

Rapport Projet mini-ML par compilation basé sur le lambda calcul

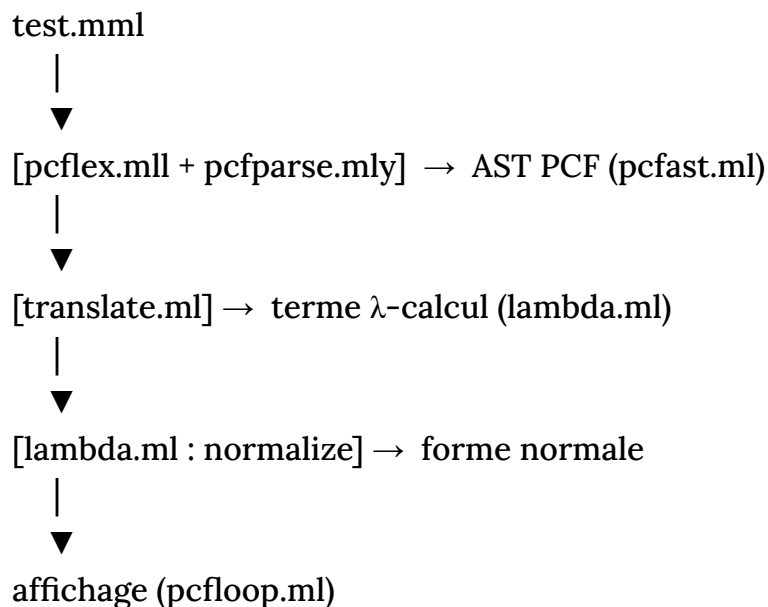
Maxime Savy - IN213

Objectif

L'objectif du projet est d'implémenter un petit langage fonctionnel, qui fonctionne en le compilant vers le λ -calcul pur. Chaque expression du langage source est traduite en un terme lambda, puis normalisée par réduction β jusqu'à sa forme normale. Voici le rapport de son implémentation.

Architecture générale

La chaîne de traitement suit un traitement linéaire en quatre étapes :



Composants réutilisés

Pour la réalisation de ce projet, j'ai décidé de réutiliser quelques composants implémentés lors du semestre.

pcflex.mll — Lexer (ocamllex)

Réutilisation de ce que nous avons fait en cours. Il tokenise le mini-ML : mots-clés (let, rec, fun, if, then, else, in), identifiants, entiers, chaînes, opérateurs arithmétiques et de comparaison...

pcfpase.mly — Parseur (ocamlyacc)

Il définit la grammaire du langage et construit l'AST via le type Pcfast.expr. Il gère les opérateurs, les applications et les syntaxes avec plusieurs paramètres.

pcfast.ml — Définition de l'AST

Il définit le type expr (14 constructeurs : Int, Bool, Fun, App, Let, Letrec, If, Binop, etc.).

Composants développés

lambda.ml — Le moteur λ -calcul

C'est le cœur théorique du projet. Il définit le type des termes et implémente la réduction β .

Type term : trois constructeurs suffisent à représenter tout le λ -calcul pur, la variable libre, l'abstraction et l'application

var_libre — calcule la liste des variables libres d'un terme. On en a besoin pour la substitution et la réduction, pour bien travailler avec les bons termes. Par exemple, pour une abstraction $\lambda x.e$, on filtre x hors des variables libres du corps, car il y est lié.

alias : génère un nouveau nom de variable via un compteur global utilisé pour éviter les captures lors de la substitution. L'utilisation de ref permet le deréférencement et donc l'incrémentatation simple du compteur.

subst x v e : Étape importante du lambda-calcul, la substitution $e[x \leftarrow v]$. On remplace x par v dans le corps de e , lorsqu'il est libre. Le cas délicat est

l'abstraction $\lambda y.e$ quand y apparaît libre dans v : on renomme y en y' avant de substituer, pour éviter qu'une variable libre de v se retrouve liée par le λ .

reduc : effectue une étape de réduction β . La règle principale est :

$(\lambda x.e) \text{ arg} \rightarrow e[x \leftarrow \text{arg}]$. La stratégie adoptée est la réduction en ordre normal, externe gauche : on réduit d'abord la position de tête, puis sous les lambdas, ce qui garantit d'atteindre la forme normale si elle existe. On s'appuie sur le type option d'OCaml pour piloter la réduction : “Some e” indique qu’une étape de réduction a été franchie, fournissant le nouveau terme, tandis que “None” signale que le terme est déjà en forme normale. Cette approche permet d'encapsuler le résultat et le statut de la réduction.

normalize : itère simplement sur le mécanisme explicité précédemment. Elle poursuit le calcul tant que *reduc* renvoie “Some e” et s'arrête dès l'obtention de “None”, marquant l'atteinte de la forme normale.

translate.ml – Traduction de PCF vers λ -calcul

Fonction récursive *translate_expr* sur l'AST PCF. Les cas implémentés sont :

Constructeur PCF	Traduction λ -calcul	Explication
Ident x	LVar x	Variable directe
Fun(x, e)	LAbs(x, e')	Abstraction directe
App($e1, e2$)	LApp($e1', e2'$)	Application directe
Let($x, e1, e2$)	$(\lambda x. e2') e1'$	Lier $x = \beta$ -réduire une abstraction
If(c, a, b)	$c' a' b'$	Booléen de Church : true $a b = a$, false $a b = b$
Int n	$\lambda f. \lambda x. f^n x$	Nombre de Church : n applications de f

La traduction de Int n construit récursivement le corps $f(f(\dots(f x)\dots))$ avec exactement n applications.

pcfast.ml – fonction *wrap_with_lib*

wrap_with_lib est une fonction qui permet de greffer les définitions de la bibliothèque sur chaque expression utilisateur avant la traduction.

pcfloop.ml — Boucle principale (modification mineure)

Le fichier fourni a été complété par une seule fonction : `print_value`, qui interprète la forme normale pour afficher `true` ou `false` quand elle correspond à un booléen, et tombe sur le λ -terme brut sinon.

La détection est structurelle et indépendante des noms de variables : `true` est la forme qui retourne son premier argument, `false` celle qui retourne le second.

lib.mml — Bibliothèque standard en mini-ML

Écrite entièrement dans le langage lui-même. Elle est chargée automatiquement au démarrage et injectée dans chaque expression évaluée via `wrap_with_lib`.

Utilisation

On écrit un programme dans un fichier `test.mml`, chaque expression terminée par `;;`

On compile et exécute :

```
make clean
```

```
make
```

```
./pcfloop test.mml
```

Pour chaque expression, l'interpréteur affiche l'AST reconnu, le λ -terme avant réduction, et le résultat final.

Conclusion

Le projet démontre qu'un langage fonctionnel assez simple peut être entièrement ramené au λ -calcul pur. Sur un gros projet, il serait même possible de faire des programmes très complexes à l'aide de celui-ci. Les booléens, les entiers et les paires ne sont pas des primitives : ce sont des fonctions ordinaires dont le comportement émerge uniquement des règles de la β -réduction. Les composants clés développés sont `lambda.ml` (moteur de réduction) et `translate.ml` (encodage de Church), tandis que le front-end lexeur/parseur a été réutilisé quasiment sans modification.